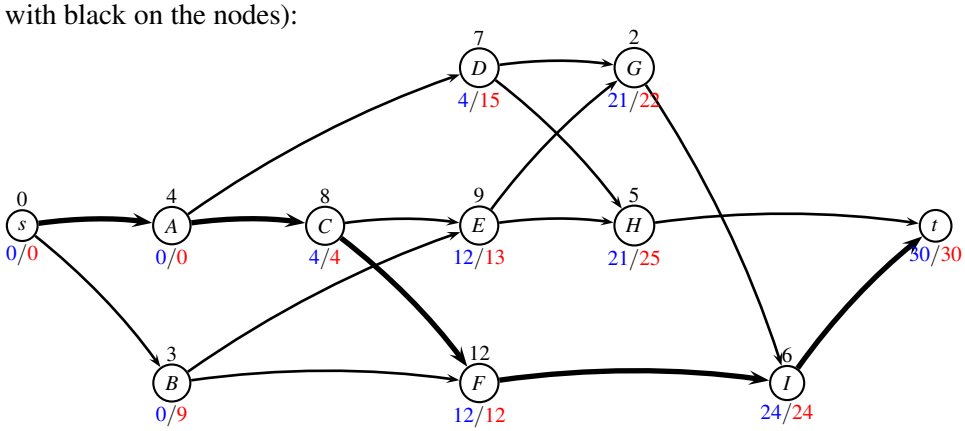
Answer, December 17, 2015.

Course name: Network Optimization **Course no:** 42115

Critical path problem

Answer 1 We insert a new node s and connect it to the first two activities A, B. We insert a new node t and connect it to the last two activities H, I. This gives the following graph (duration of activities is indicated with black on the nodes):



Now we solve the problem

- *Forward procedure:* Starting at time zero, calculate the earliest time U_j each job j can be started $U_j = \max_{(i,j)\in E}\{U_i + p_i\}$. (marked with blue, above)
- *Makespan:* $T = U_t$.
- *Backward procedure:* Starting at time equal to the makespan, calculate the latest time V_j each job j can be started so that this makespan is realized. $V_i = \min_{(i,j)\in E}\{V_j - p_i\}$ (marked with red, above)

The critical path schedule solution for this project is given below:

Activity	Duration	Earliest start time U_j	Latest start time V_j
s	0	0	0
A	4	0	0
B	3	0	9
C	8	4	4
D	7	4	15
E	9	12	13
F	12	12	12
G	2	21	22
H	5	21	25
I	6	24	24
t	0	30	30

The makespan is $T = 30$. ■

Answer 2 The critical path is found for activities where earliest start time U_j is equal to the latest start time V_j in the above table. This means projects A, C, F, I. The critical path is marked with bold edges in the above graph. ■

Fixed charge transportation problem

Answer 3 We have the following possible patterns

	S_1			S_2		S_3		
T_1	3	2	1	1	0	2	1	0
T_2	0	1	2	0	1	0	1	2
transp. cost	12	11	10	5	2	14	8	2
fixed charges	5	10	10	5	5	5	10	5
sum of costs	17	21	20	10	7	19	18	7

This leads to the Dantzig-Wolfe decomposed model:

```
minimize
17p11+21p12+20p13+10p21+ 7p22+19p31+18p32+ 7p33
subject to
 3p11+ 2p12+ 1p13+ 1p21+ 0p22+ 2p31+ 1p32+ 0p33 = 4   (u1)
 0p11+ 1p12+ 2p13+ 0p21+ 1p22+ 0p31+ 1p32+ 2p33 = 2   (u2)

  p11+  p12+  p13                                  = 1   (v1)
                   p21+  p22                       = 1   (v2)
                                 p31+  p32+  p33 = 1   (v3)

binary p11 p12 p13 p21 p22 p31 p32 p33
end
```

Note: In principle, one can add a pattern p_{14} for S_1 transporting $(0,3)$ but this pattern will never be used. However it does not make any harm. ■

Answer 4 The greedy solution uses patterns $p_{13} = (1,2), p_{21} = (1,0), p_{31} = (2,0)$. ■

Answer 5 Let a_{ip} be the coefficients in the two first lines of the pattern formulation, and b_{ip} be the coefficients in the last three lines. Reduced cost for pattern p is then calculated as

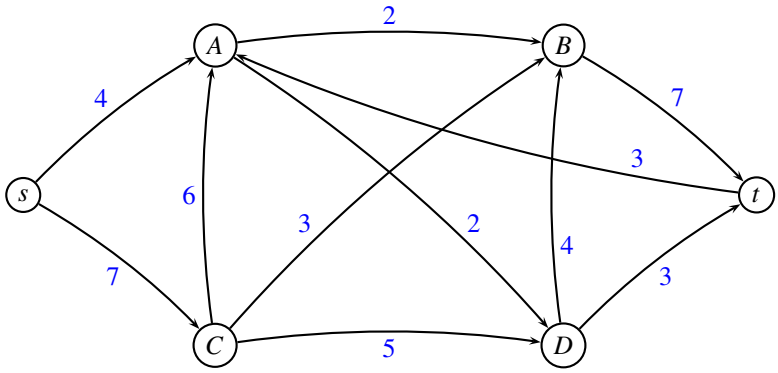
$$\overline{c}_p = c_p - \sum_i a_{pi} u_i - \sum_i b_{pi} v_i$$

where we have $u_1 = 0$, $u_2 = 10$, $v_1 = 0$, $v_2 = 10$, $v_3 = 19$. This gives the following reduced costs (we do not need to calculate reduced costs of paths which already are present in the master problem)

pattern p	c_p	$\sum_i a_{pi} u_i$	$\sum_i b_{pi} v_i$	$\overline{c}_p$
p_{11}	17	0	0	17
p_{12}	21	10	0	11
p_{22}	7	10	10	-13
p_{32}	18	10	19	-11
p_{33}	7	20	19	-32

Pattern p_{33} has the most negative reduced cost. Notice that in pattern p_{33} the sum $\sum_i a_{pi}u_i = 20$ since we are sending two units. ■

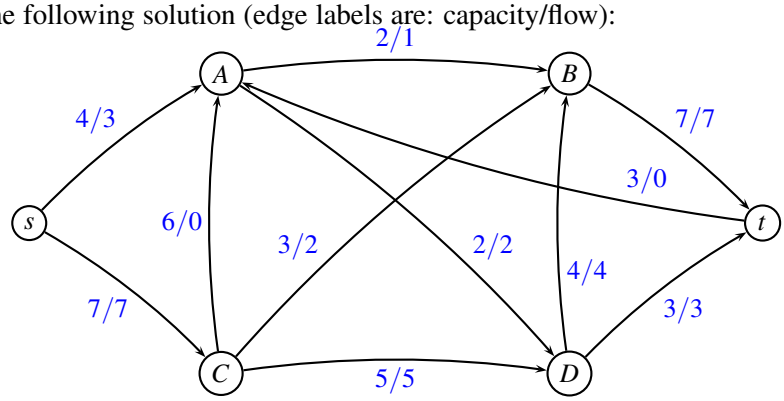
Maximum flow



Answer 6 We use the paths:

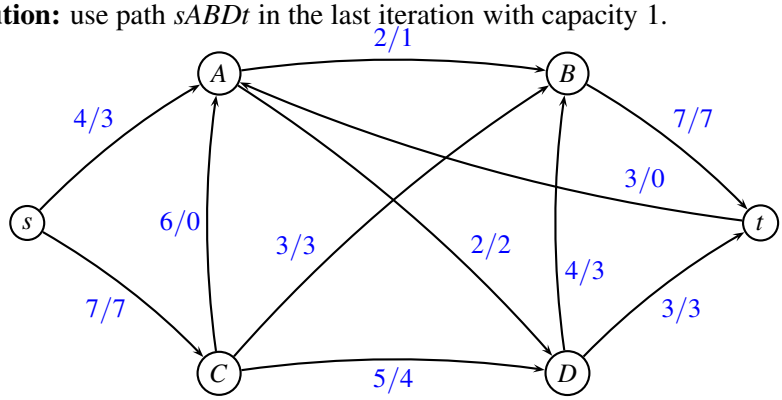
path	capacity
sCDBt	4
sCBt	3
sADt	2
sABCDt	1

This results in the following solution (edge labels are: capacity/flow):



The maximum flow is 10.

Alternative solution: use path *sABDt* in the last iteration with capacity 1.



■

Answer 7 A minimum cut is $\{s,A,B,C,D\}$ and $\{t\}$. The capacity of the cut is $7+3=10$. This corresponds to the solution found above. ■

Minimum cost flow problem

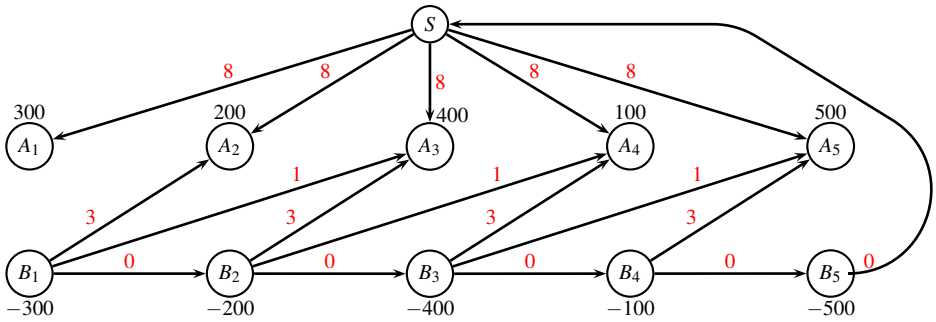
Answer 8 In order to formulate the problem as a *minimum cost flow problem*, we introduce two nodes for each day. Node A_i represents the start of day i, and B_i represents the end of the day. If the demand of napkins at day i is d_i, then node A_i has a demand d_i and node B_i has demand $-d_i$.

At the end of day i we can wash napkins in two possible ways. At cost 3 they can be ready at the beginning of day $i+1$, or at cost 1 they can be ready at day $i+2$. This means that each node B_i is connected to node A_{i+1} at cost 3, and to node A_{i+2} at cost 1. The capacity of the edges is infinite.

We can also buy napkins every day at cost 8. We therefore introduce a source s representing buying new napkins. Every node A_i is connected with an edge from s at a cost of 8 with infinite capacity.

Finally, we need to get rid of all napkins not washed. This can e.g. be modeled by connecting node B_i to node B_{i+1} at cost 0, and from node B_5 to node s at cost 0. The edge capacities are infinte.

This gives the following graph formulation:



All edge capacities are infinite.

Alternative solution: Add a terminal node t for all napkins not washed. Connect all B_i nodes to t at cost 0. Give s a demand of $-\sum d_i = -1500$ and t a demand of $\sum d_i = 1500$. Finally, add an extra edge from s to t having cost 0. In this way a sufficient amount of new napkins is available at s, and when napkins are not used any more they disappear at t. Excess napkins which are not used in the catering are sent directly from s to t at cost 0. ■

Arbitrage opportunity

	USD	EUR	DKK	CHF	GBP
USD	1.00	.88	6.54	.96	.65
EUR	1.14	1.00	7.46	1.09	.74
DKK	.15	.13	1.00	.15	.10
CHF	1.05	.92	6.84	1.00	.68
GBP	1.54	1.35	10.05	1.47	1.00

Answer 9 The problem can be formulated by use of a complete directed graph, where each node corresponds to a currency, and edge weights correspond to the exchange rate. We are looking for a cycle where the product of the edges is larger than 1.00.

To solve this problem we first take the logarithm to the exchange rates, and then we multiply by -1 to change maximization to minimization. In this way the problem becomes to find a *negative cost cycle* in the graph. This can be done by use of *Bellman-Ford*.

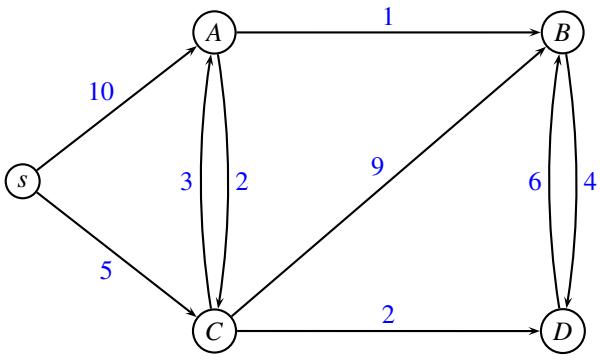
Note: It is not possible to solve the above problem as a *shortest path problem* since if we have a negative cost cycle, the distances are not welldefined, and the algoritm may not terminate.

Note: It should not be necessary to add a source s and a sink t to the graph. Indeed, if all nodes (currencies) are connected to the source s, and also all nodes (currencies) are connected to the sink t then you may risk that you start with one currency and end up with a different currency! It is always easy to exchange e.g. GBP to DKK so that you get more units, but the overall value will not necessarily be better. ■

The following table illustrates the transformation. Each entry is the $-\log(e_{ij})$ of the original exchange rate e_{ij}. Each currency will be a node, and the edge weights are given in the table.

	USD	EUR	DKK	CHF	GBP
USD	0	0,05558	-0,8156	0,0177	0,1871
EUR	-0,0569	0	-0,8727	-0,0374	0,1308
DKK	0,8239	0,886	0	0,8239	1
CHF	-0,0212	0,0362	-0,835	0	0,167
GBP	-0,1875	-0,1303	-1,002	-0,167	0

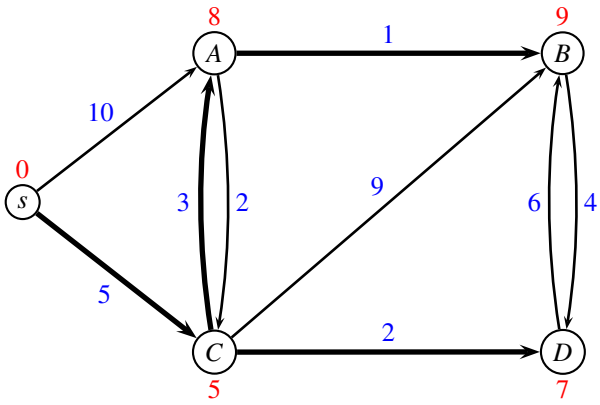
Shortest path



Answer 10 The following table states for each iteration which node is considered, and what the so far best distances are to each node. When a node has been considered, the distance is not changed any more.

considered	distance				
node	s	A	B	C	D
init	0	∞	∞	∞	∞
s		10		5	
C		8	14		7
D			13		
A			9		
B					

The shortest path tree is marked with fat edges below (shortest distance marked with red):

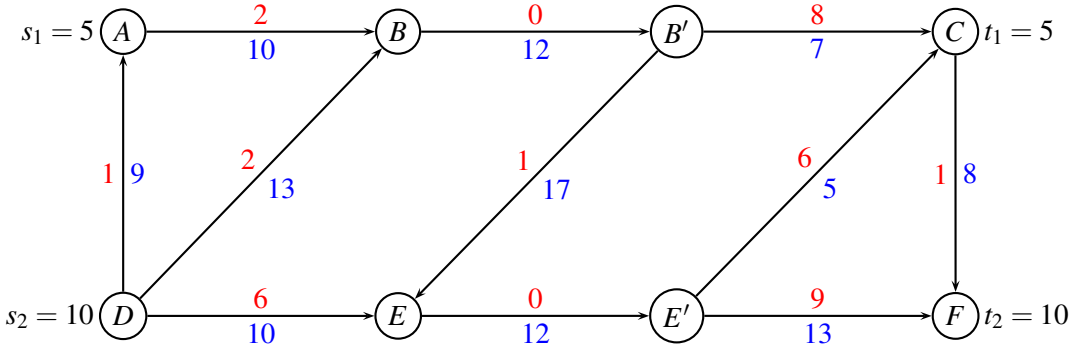


■

Multi-commodity Flow

Answer 11 Every node v with a capacity c_v is split into two nodes v, v'. All ingoing edges are connected to v while all outgoing edges are connected to v'. A new edge (v, v') is added with capacity c_v and cost 0.

This gives us the following standard multi-commodity flow problem



■

Answer 12 If all demands are to be transported unsplittable, we notice that commodity 1 can either use path ABC or path $ABEC$.

Due to the capacity constraints in nodes B and E commodity 1 and commodity 2 cannot pass through nodes B and E at the same time. Therefore, if commodity 1 chooses path $ABEC$ it will not be possible to send commodity 2 through the network. We can therefore conclude that commodity 1 will use path ABC.

Now, commodity 2 can only use path DEF since no other paths have sufficient capacity.

This gives the solution: Commodity 1 use path ABC and commodity 2 use path DEF. The total cost is $5 \cdot (2+8) + 10 \cdot (6+9) = 50 + 150 = 200$. ■